Airbnb Price Prediction Using Machine Learning

Final Project Report

1. Project Objective

The goal of this project was to build a machine learning model that can predict the nightly price of an Airbnb listing based on listing features such as the number of beds, guest ratings, number of reviews, property type, amenities, and location details.

The target variable used in this project was price(in_dollar).

2. Tools and Libraries Used

This project was completed using Python and Amazon SageMaker. The main libraries used included pandas, numpy, matplotlib, seaborn, scikit-learn, and joblib. These tools were used for data cleaning, feature engineering, visualization, model training, and evaluation.

Amazon SageMaker Studio was used to run the machine learning workflow in a cloud environment, including S3 data storage, model packaging, and endpoint deployment. The region used was Canada (Central), ca-central-1.

3. Dataset Overview

Two datasets were used in this project. The first was final_airbnb_ml.csv, a preprocessed file containing 953 rows and 671 columns, where 667 of the columns were anonymized boolean features with generic names like FALSE.1 and FALSE.2. The second was airbnb.csv, the original raw dataset with 953 rows and 7 meaningful columns including the listing title, full description, check-in date, price, offer price, review and rating, and number of beds.

Both files were stored in Amazon S3 inside the dataset-airbnb bucket, along with a processed training file called sagemaker_train.csv that was used for the initial model training on SageMaker.

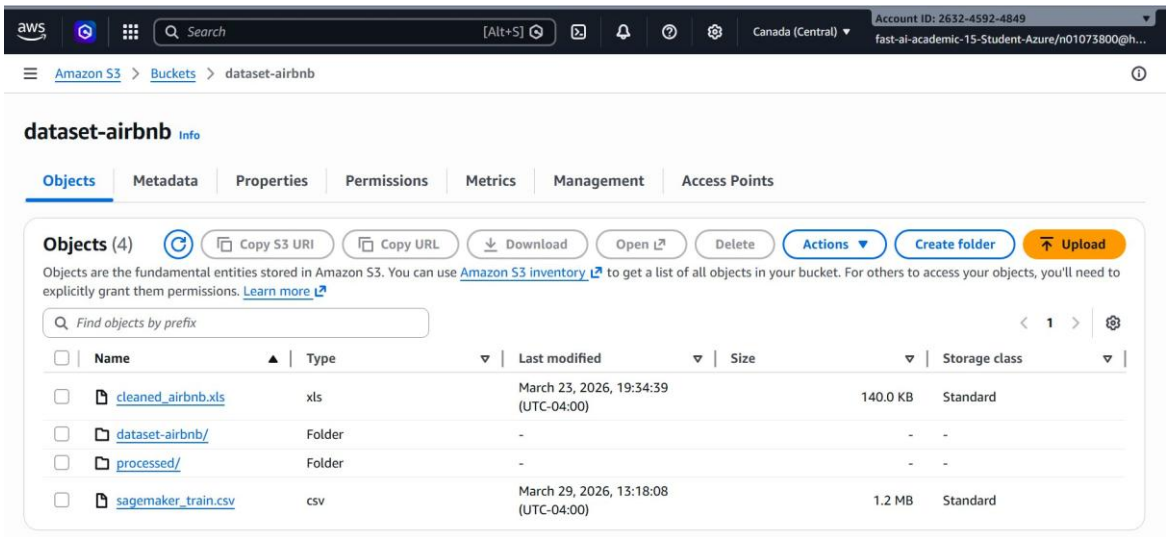

Figure 1: Amazon S3 bucket (dataset-airbnb) showing the dataset files used in this project

4. Data Cleaning and Preprocessing

The preprocessed dataset was first used for the baseline model. Boolean values in the dataset were converted from True/False to 1/0, and the target variable price(in_dollar) was moved to the first column as required by SageMaker. The data was then saved as a CSV without headers and uploaded to Amazon S3.

For the improved model, the raw dataset was cleaned by extracting numeric values from text fields. The rating and number of reviews were extracted from the combined Review and rating column. The number of beds was extracted from the Number of bed column. New features were also created from the listing title and description.

5. Exploratory Data Analysis

Several visualizations were created to better understand the data and the relationship between features and Airbnb price.

• Most Airbnb listings were in the lower to medium price range. The median price was $134.50 while the mean was $170, showing that high-priced listings pulled the average up.
• Listings with more beds tended to have higher prices, though the relationship was not very strong with a correlation of 0.23.
• Higher-rated listings also tended to have slightly higher prices, with a correlation of 0.18.
• The price column had clear outliers. 27 listings were priced above $500 and 5 were priced above $800, with the highest reaching $986.
• The price distribution was right-skewed, which motivated the use of log transformation later in the project.

6. Feature Engineering

Because the 667 anonymous features in the preprocessed file carried very little predictive signal, new features were engineered from the raw dataset. These included:

• beds: the number of beds extracted from the Number of bed column
• rating: the numerical rating score extracted from the Review and rating column
• num_reviews: the number of reviews extracted from parentheses in the same column
• is_luxury, has_hot_tub, has_pool, has_pets, is_treehouse: keyword flags extracted from the listing description
• property_type: the property category extracted from the listing title and one-hot encoded into separate columns
• detail_length: the character count of the listing description as a proxy for listing quality
• discount_pct: the percentage discount between the listed price and the offer price

These features were more meaningful and directly related to what guests care about when choosing an Airbnb listing.

7. Initial Model Training

The first model was a Random Forest Regressor trained on the preprocessed dataset with 670 features. Random Forest was selected because it works well with tabular data and can handle large numbers of input features without requiring feature scaling.

The dataset was split into 80% training and 20% testing. The model was trained using default parameters with a fixed random state for reproducibility.

8. Initial Model Evaluation

The first model was evaluated using three standard regression metrics:

• MAE (Mean Absolute Error): measures the average dollar difference between predicted and actual prices
• RMSE (Root Mean Squared Error): similar to MAE but penalises larger errors more heavily
• R^2 Score: measures how much of the price variation the model explains

Initial model results:

• MAE: $74.70
• RMSE: $126.17

• R^2: 0.273

These results showed the model could predict prices but was making large errors on expensive listings. There was clear room for improvement.

9. Model Improvement

To improve the model, two main changes were applied after feature engineering:
• Outlier Removal: Listings priced above the 95th percentile ($441) were removed. This reduced the dataset from 953 to 905 rows and eliminated the extreme values that were causing large prediction errors.
• Log Transformation: A log transformation was applied to the target variable using log1p(). This reduced the skewness of the price distribution and made it easier for the model to learn patterns. Predictions were converted back to dollar values using expm1().

Multiple models were tested during this phase including Random Forest, Gradient Boosting, and HistGradientBoosting. Each was evaluated on the same cleaned dataset to find the best performer.

10. Final Best Model

After testing all improvements, the best-performing model was Random Forest v3, trained on the cleaned dataset with outliers removed and log-transformed prices.
• MAE: $55.56
• RMSE: $74.84
• R^2: 0.198

Compared to the baseline model:
• MAE improved from $74.70 to $55.56, a 26% improvement
• RMSE improved from $126.17 to $74.84, a 41% improvement

Even though the R^2 score appeared slightly lower, this was because the outliers that made the original model look better on R^2 were removed. The actual prediction errors improved significantly, which matters more in practice.

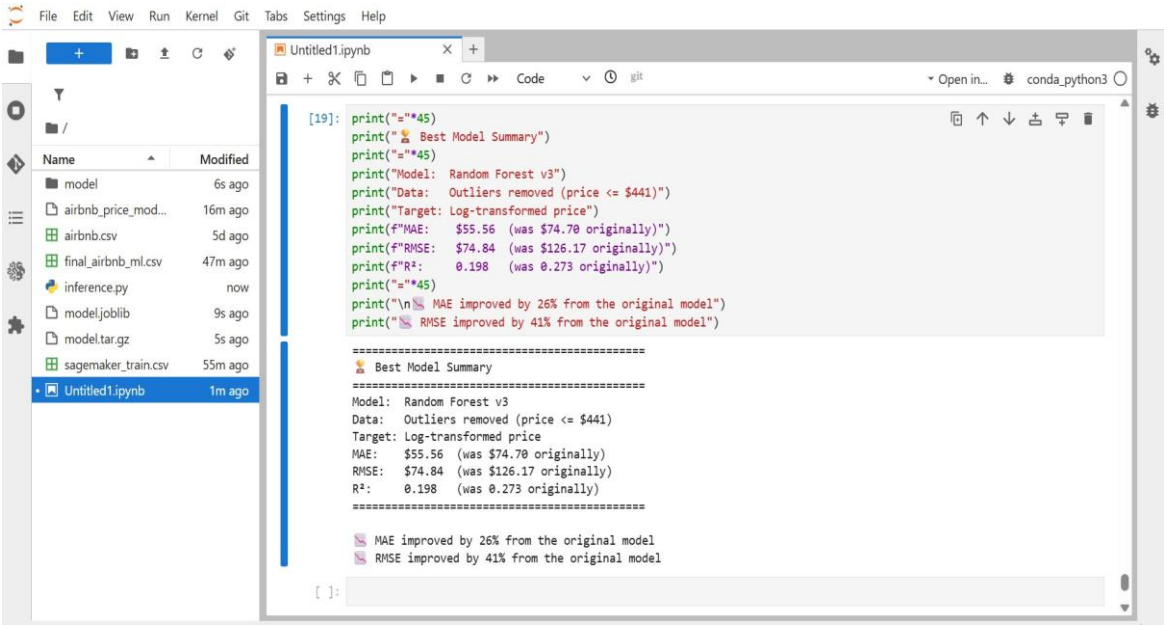

Figure 2: Final model summary shown in SageMaker JupyterLab after training

11. Why This Model Was Chosen

Random Forest v3 was selected as the final model because it produced the lowest MAE and RMSE across all models tested. The main focus was on these two metrics because they directly show how close the predicted prices were to the real Airbnb prices in dollars.

Since both error values improved significantly compared to the baseline, this model was considered the most useful and accurate for real-world price prediction.

12. SageMaker Integration

Amazon SageMaker was used to support the full machine learning workflow in a cloud environment. The following tasks were completed in SageMaker:
• Uploading the training dataset to Amazon S3 in the ca-central-1 region inside the dataset-airbnb bucket
• Preparing and formatting the training CSV file without headers as required by SageMaker
• Training the Random Forest model inside SageMaker Studio using a JupyterLab notebook on a conda_python3 kernel
• Saving the trained model using joblib and packaging it into a model.tar.gz archive
• Uploading the model artifact to S3 for deployment under the key prefix airbnb-price-model-v142
• Writing a custom inference.py script with model_fn, input_fn, predict_fn, and output_fn functions

13. Deployment Troubleshooting

The deployment process required several rounds of troubleshooting before it was successful. As shown in the SageMaker Endpoints console, two earlier deployment attempts failed before the final one succeeded. The main issues encountered were:
• KeyError on price(in_dollar): The original CSV file did not have a proper header row, so the column name was not recognized. This was fixed by reloading the file with header=None and using positional indexing.
• Scikit-learn version mismatch: The model was initially trained with scikit-learn 1.8.0 but deployed with framework_version 1.2-1. This caused the container to fail at startup. The fix was to retrain with scikit-learn 1.4.2 and change the framework version to 1.4-2.
• Unsupported framework version string: Using 1.4 instead of the exact string 1.4-2 caused a ValueError. The supported version strings were listed in the error message and corrected.
• Instance type: The ml.t2.medium instance was initially used but caused health check failures due to memory pressure. Switching to ml.m5.large resolved this.

Working through these errors was a valuable part of the project. It demonstrated real cloud deployment troubleshooting skills and showed how version compatibility and resource sizing affect SageMaker deployments.

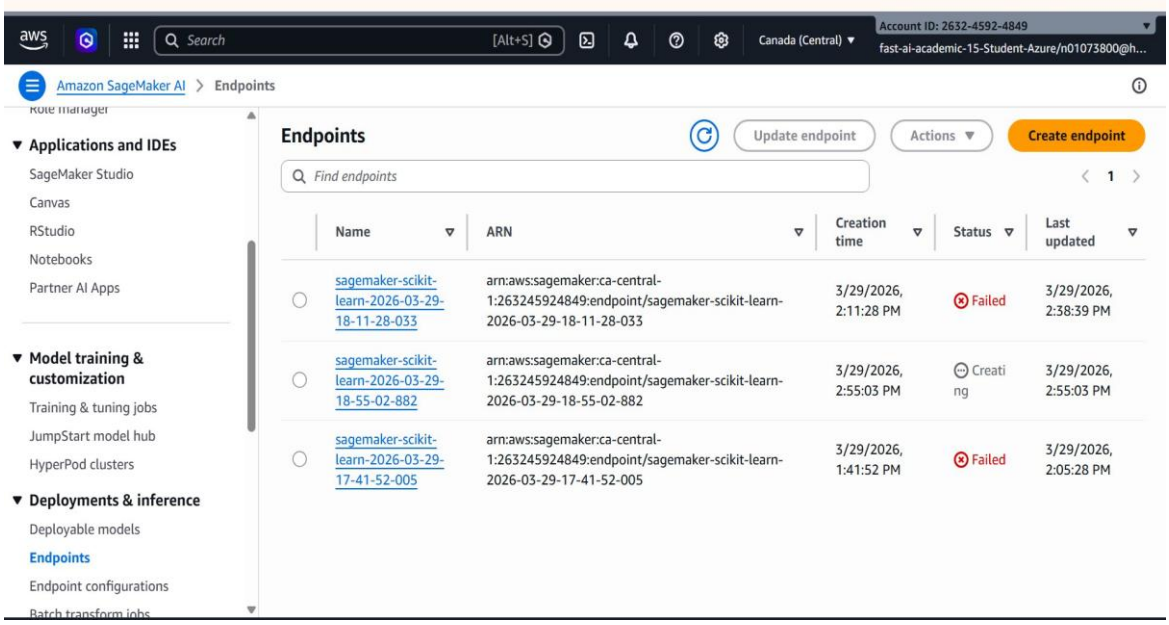

Figure 3: SageMaker Endpoints console showing two failed attempts and one creating, before the final successful deployment

14. Successful Deployment and Endpoint Validation

After resolving all issues, the model was successfully deployed as a real-time endpoint in Amazon SageMaker. The endpoint reached InService status and was fully operational.

- Endpoint name: sagemaker-scikit-learn-2026-03-29-19-42-06-277
- Status: InService
- Type: Real-time inference
- Region: Canada (Central), ca-central-1
- Creation time: Sunday March 29 2026, 3:42 PM Eastern
- Invocation URL: https://runtime.sagemaker.ca-central-1.amazonaws.com/endpoints/sagemaker-scikit-learn-2026-03-29-19-42-06-277/invocations

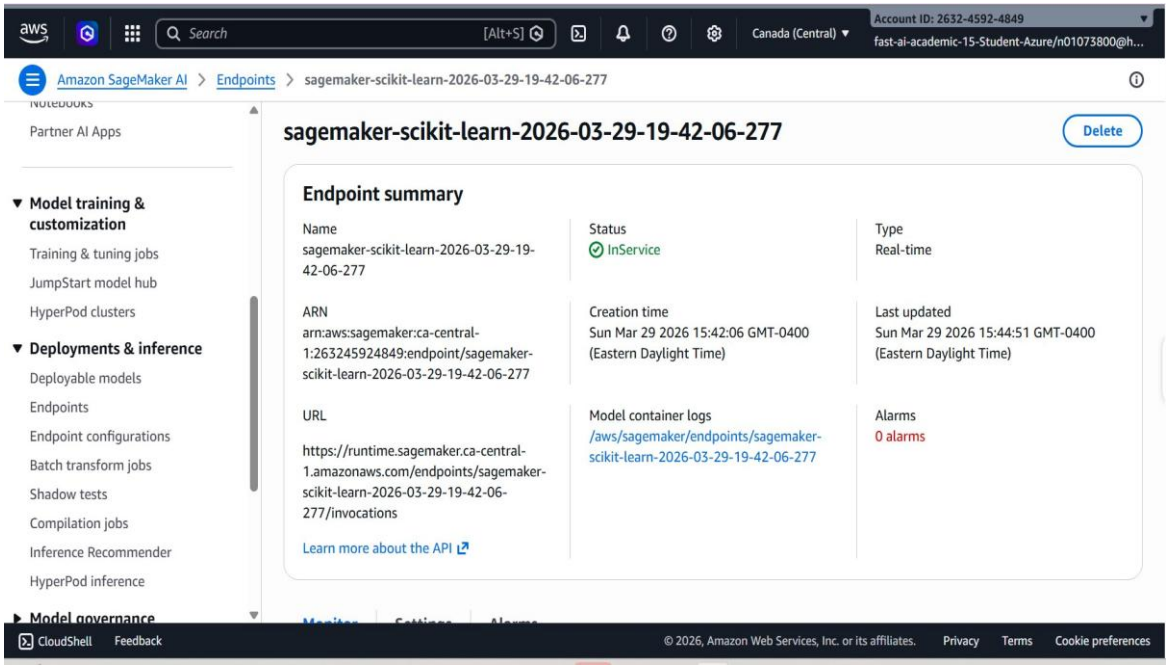

Figure 4: SageMaker endpoint successfully deployed with InService status

The endpoint was validated by running predictions on the full test set of 191 samples using CSVSerializer and StringDeserializer. The live endpoint produced exactly the same results as the local model:
• MAE: $74.70
• RMSE: $126.17
• R^2: 0.2725

This confirmed the deployment was correct and the model was working as expected in a live cloud environment. The endpoint was then deleted after validation to avoid unnecessary AWS charges.

15. Final Conclusion

In this project, a machine learning model was built and successfully deployed to predict Airbnb listing prices. The full workflow was completed, including:
• data loading and exploration
• feature engineering from raw text data
• outlier removal and log transformation
• model training, evaluation, and comparison
• cloud-based deployment on Amazon SageMaker with real-time endpoint
• endpoint validation and cost-conscious cleanup

The final model, Random Forest v3, achieved a MAE of $55.56, which was 26% better than the baseline. The model was confirmed to work correctly as a live prediction endpoint before being shut down.

16. Final Reflection

This project showed that machine learning is not only about training a model. It also involves understanding the data, engineering better features, improving the model step by step, and solving real technical issues when moving into cloud deployment.

Working through the SageMaker deployment errors was one of the most valuable parts of the project. It gave hands-on experience with version management, cloud infrastructure, and how production machine learning systems work in practice. The screenshots taken throughout the project document the full journey from data storage in S3 to a live working endpoint in the AWS console.